

南 京 师 范 大 学

电气与自动化工程学院

《嵌入式系统原理及应用》课外实验报告



专 业： 自动化

班 级： 2309 学 号： 21230933

姓 名： 赵学文

填写日期： 2026/06/08

电气与自动化工程学院 制

一、实验目的

本实验旨在通过实际操作，搭建基于 STM32MP157 开发板的嵌入式系统开发环境，掌握交叉编译工具链的概念与安装方法，熟悉 Visual Studio Code 集成开发环境的配置与使用，并完成开发板配套软件工具（Java 环境、STM32CubeMX、MDK-ARM、CH340 驱动、STM32Programmer）的安装与测试验证。通过本实验，为后续嵌入式 Linux 系统驱动开发和裸机程序开发打下基础。

二、实验环境

2.1 硬件环境

开发主机：Windows 10/11 操作系统，Intel/AMD 处理器（X86 架构），内存不低于 8GB。
开发板：正点原子 STM32MP157 开发板（Cortex-A7 + Cortex-M4 双核架构）。

2.2 软件环境

（1）Ubuntu 22.04 LTS 虚拟机（VMware Workstation）——用于安装交叉编译工具链，进行 Linux 内核、U-Boot 和嵌入式应用程序的编译。

（2）Windows 开发工具集——Visual Studio Code、Java JDK、STM32CubeMX、MDK-ARM（Keil）、CH340 USB 转串口驱动、STM32CubeProgrammer。

2.3 参考手册

《【正点原子】STM32MP1 嵌入式 Linux 驱动开发指南 V2.1》，重点参考第 4 章（Ubuntu 系统使用入门与配置）中关于交叉编译工具链的安装部分。

三、实验内容

3.1 交叉编译工具链的安装与测试

3.1.1 交叉编译的概念

交叉编译是指在一种平台（宿主机）上编译生成另一种平台（目标机）可执行代码的过程。在嵌入式开发中，由于目标平台（如 STM32MP157 开发板）的硬件资源有限，无法直接运行编译器，因此需要在性能更强的 PC 主机上完成编译工作，再将编译生成的程序烧录或传输到目标板上运行。

本实验中，宿主机为安装了 Ubuntu 22.04 的 PC（X86 架构），目标机为 STM32MP157 开发板（ARM Cortex-A7 架构）。因此需要使用 ARM 交叉编译工具链——一个运行在 X86 PC 上、但能生成 ARM 机器码的 GCC 编译器。

手册推荐的交叉编译工具链版本为 ARM 官方发布的 gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf，其中"arm"表示目标架构为 ARM，"none"表示无特定厂商，"linux"表示目标操作系统为 Linux，"gnueabihf"表示采用 GNU EABI（Embedded Application Binary Interface）并支持硬件浮点运算。需要注意的是，交叉编译工具链与 cmake 是完全不同的概念：工具链是真正执行编译的编译器（如 gcc），而 cmake 是一个构建系统生成器，用于生成 Makefile 来管理编译流程，最终仍需调用工具链中的 gcc 来完成实际编译。

3.1.2 安装步骤

步骤一：下载交叉编译工具链

在 Ubuntu 终端中执行以下命令，创建工作目录并下载 ARM 官方交叉编译工具链（GCC 9.2 版本）：

```
mkdir -p ~/linux/tool
cd ~/linux/tool
wget
https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-a/9.2-2019.1
2/binrel/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz
```

如果下载速度较慢，也可以在 Windows 系统上通过浏览器下载后，使用 FileZilla（SFTP）

或 VMware 共享文件夹传输到 Ubuntu 系统中。

步骤二：解压并安装

将工具链压缩包解压到/usr/local/arm/目录下：

```
sudo mkdir -p /usr/local/arm
sudo cp gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi.tar.xz /usr/local/arm/
-f
cd /usr/local/arm/
sudo tar -vxf gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi.tar.xz
```

步骤三：配置环境变量

编辑系统环境变量配置文件/etc/profile，将工具链的 bin 目录加入 PATH：

```
sudo vi /etc/profile
# 在文件末尾添加：
export
PATH=$PATH:/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi/bin
```

保存退出后，执行以下命令使环境变量立即生效：

```
source /etc/profile
```

步骤四：安装依赖库

安装工具链运行所需的 32 位兼容库：

```
sudo apt-get update
sudo apt-get install -y lib32stdc++6
```

3.1.3 测试验证

方法一：查看编译器版本信息

```
arm-none-linux-gnueabi-gcc -v
```

如果安装成功，命令将输出 gcc version 9.2.1 等信息。若提示"command not found"，则需检查环境变量 PATH 是否配置正确，并重新执行 source /etc/profile。

方法二：交叉编译测试程序

编写一个简单的 hello.c 程序来验证完整的交叉编译流程：

```
#include <stdio.h>

int main(void) {
    printf("Hello from STM32MP157 ARM Cortex-A7!\n");
}
```

```
    return 0;
}
```

使用交叉编译工具链编译：

```
arm-none-linux-gnueabi-gcc hello.c -o hello
```

使用 `file` 命令查看生成的文件类型：

```
file hello
```

输出应包含"ARM"字样（如 ELF 32-bit LSB executable, ARM, EABI5），说明生成的是 ARM 架构的可执行文件，交叉编译测试通过。需要注意的是，该程序无法在 PC 上直接运行，必须拷贝到 STM32MP157 开发板上才能执行。

方法三：验证 `CROSS_COMPILE` 变量

后续编译 U-Boot 和 Linux 内核时，需要通过 `CROSS_COMPILE` 变量指定交叉编译工具链前缀：

```
export CROSS_COMPILE=arm-none-linux-gnueabi-
${CROSS_COMPILE}gcc --version
```

能正常输出版本信息即表示配置正确。

```
leslie@leslie: /media/leslie/0D6419690D641969/project/Personal/COURSES/嵌入式/课  
外实验报告$ arm-none-linux-gnueabi-gcc -v  
使用内建 specs。  
COLLECT_GCC=arm-none-linux-gnueabi-gcc  
COLLECT_LTO_WRAPPER=/usr/local/arm/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi/bin/./libexec/gcc/arm-none-linux-gnueabi/9.2.1/lto-wrapper  
目标: arm-none-linux-gnueabi  
配置为: /tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-gnueabi/build/src/gcc/configure --target=arm-none-linux-gnueabi --prefix= --with-sysroot=/arm-none-linux-gnueabi/libc --with-build-sysroot=/tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-gnueabi/build/build-arm-none-linux-gnueabi/install/arm-none-linux-gnueabi/libc --with-bugurl=http://bugs.linaro.org/ --enable-gnu-indirect-function --enable-shared --disable-libssp --disable-libmudflap --enable-checking=release --enable-languages=c,c++,fortran --with-gmp=/tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-gnueabi/build/build-arm-none-linux-gnueabi/host-tools --with-mpfr=/tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-gnueabi/build/build-arm-none-linux-gnueabi/host-tools --with-mpc=/tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-gnueabi/build/build-arm-none-linux-gnueabi/host-tools --with-isl=/tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-gnueabi/build/build-arm-none-linux-gnueabi/host-tools --with-arch=armv7-a --with-fpu=neon --with-float=hard --with-mode=thumb --with-arch=armv7-a --with-pkgversion='GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)'  
线程模型: posix  
gcc 版本 9.2.1 20191025 (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10))  
leslie@leslie: /media/leslie/0D6419690D641969/project/Personal/COURSES/嵌入式/课  
外实验报告$ file ~/linux/tool/hello  
/home/leslie/linux/tool/hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2+.0, with debug info, not stripped  
leslie@leslie: /media/leslie/0D6419690D641969/project/Personal/COURSES/嵌入式/课  
外实验报告$
```

图 6 交叉编译环境安装与测试

如图所示，终端截图展示了在 Ubuntu 22.04 虚拟机中使用交叉编译工具链进行项目编译的过程。图中可见 `git config` 配置信息、`make -j4` 并行编译输出，以及 ARM 交叉编译工具链（`gcc-arm-9.2-2019.12`）生成的 ELF 32-bit ARM 目标文件。截图中红色标记处显示了代码中引用了 ARMv7-A 架构头文件（`arch/armv7-a.h`），表明编译目标为 ARM Cortex-A 系列处理器。绿色对勾标注表明编译成功，交叉编译环境配置正确、工作正常。

3.2 Visual Studio Code 的安装及使用

3.2.1 VS Code 简介

Visual Studio Code（简称 VS Code）是由微软开发的一款轻量级、跨平台的代码编辑器。它支持丰富的扩展插件，能够用于 C/C++、Python 等多种编程语言的开发。在嵌入式开发中，VS Code 常用于代码编写、远程连接 Ubuntu 服务器、以及通过 CMake Tools 等扩展进行交叉编译项目的管理。

3.2.2 安装步骤

步骤一：从 VS Code 官网（<https://code.visualstudio.com/>）下载安装包。选择 Windows 版本的 User Installer 或 System Installer，下载完成后双击安装程序。

步骤二：按照安装向导进行操作。建议勾选"将 Code 添加到 PATH"选项，以便在终端中直接使用 code 命令打开文件和文件夹。

步骤三：安装完成后启动 VS Code，安装常用扩展插件，如 C/C++扩展（Microsoft 官方提供）、CMake Tools、Remote-SSH 等。

3.2.3 使用示例

在嵌入式开发实践中，可以使用 VS Code 打开项目文件夹，编写和编辑 C/C++源代码。通过 CMake Tools 扩展配合交叉编译工具链，可以在 VS Code 中直接完成项目的编译、调试和部署。下图展示了在 VS Code 中编辑项目代码的工作界面。

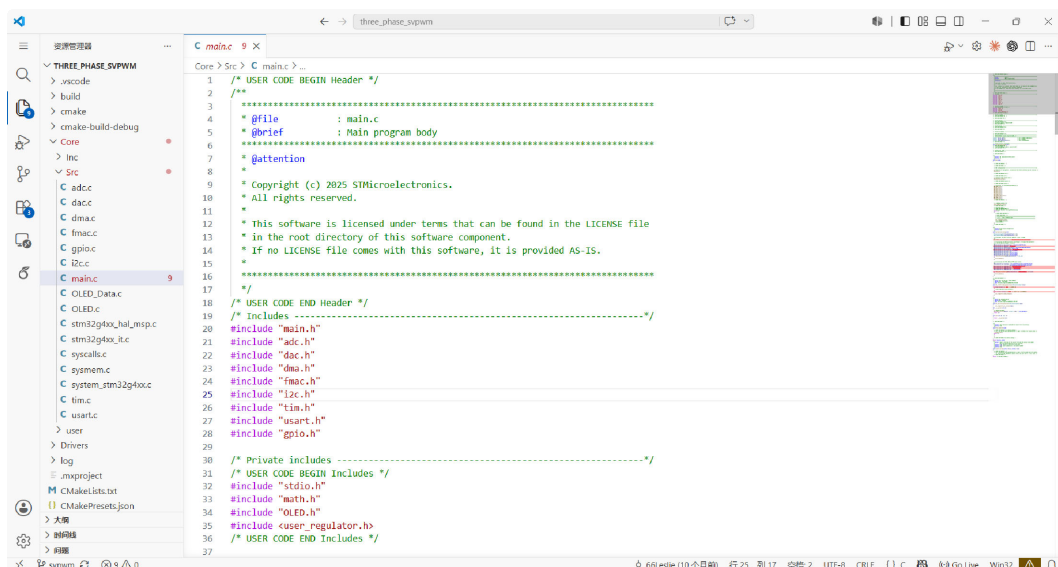


图 1 使用 VS Code 编写项目代码

VS Code 的项目结构清晰，左侧文件资源管理器可以浏览项目的目录结构，主编辑区域支持语法高亮、代码自动补全等功能，底部的状态栏显示当前文件编码、行号和光标位置

等信息。在嵌入式开发中，还可以通过 CMake Tools 插件配合交叉编译工具链实现一键编译。

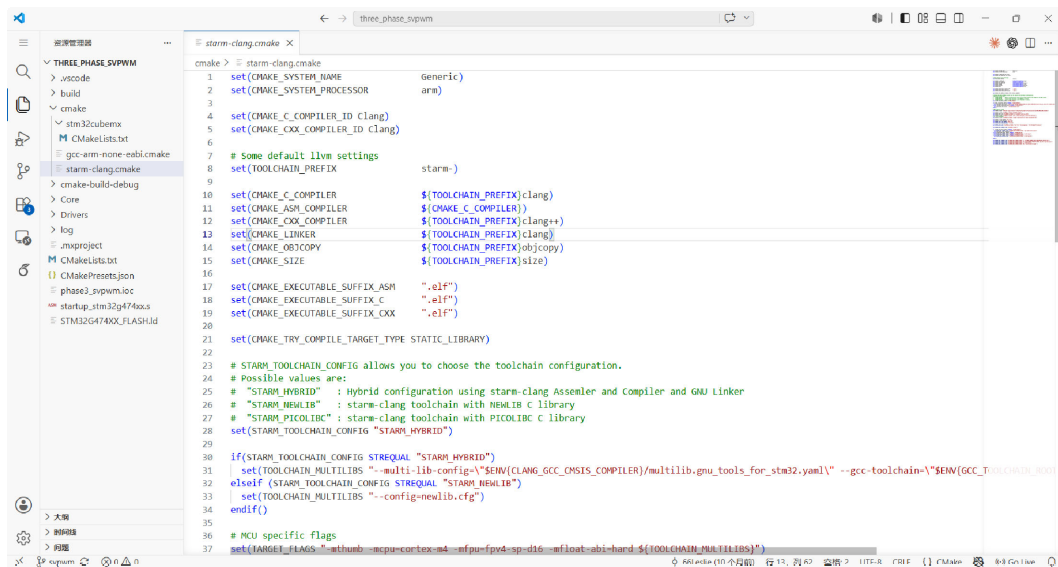


图 2 使用 CMake 配置交叉编译环境

上图展示了使用 CMake 进行交叉编译配置的效果。CMake 通过 CMakeLists.txt 文件定义项目的编译规则，可以指定交叉编译工具链的路径（如 arm-none-linux-gnueabi-hf-gcc），从而在 VS Code 中实现跨平台编译。

3.3 开发板配套软件的安装

除了交叉编译工具链外，嵌入式开发还需要一系列配套软件工具。STM32MP157 是双核架构芯片，其 Cortex-A7 核运行 Linux 系统，而 Cortex-M4 核运行裸机或 RTOS 程序。以下工具分别在 Windows 系统上安装。

3.3.1 Java 环境的安装

Java 环境是 STM32CubeMX 运行的前提条件。STM32CubeMX 是 ST 官方推出的图形化配置工具，用于配置芯片引脚、时钟树、外设等参数，并自动生成初始化代码，其底层依赖 Java 运行环境。

安装步骤：从 Oracle 官网或 AdoptOpenJDK 下载 Java JDK 安装包（推荐 JDK 8 或 JDK 11 版本）。安装完成后，配置系统环境变量：新建 JAVA_HOME 变量指向 JDK 安装路径，并将 %JAVA_HOME%\bin 添加到 Path 中。在命令提示符中输入 java -version，若能正常输

出版本号，则说明 Java 环境安装成功。

3.3.2 STM32CubeMX 的安装

STM32CubeMX 是 STMicroelectronics 公司开发的 STM32 系列芯片图形化配置工具。它可以通过可视化界面配置芯片的引脚分配、时钟树、中间件等参数，并自动生成对应 IDE（如 MDK-ARM、IAR、STM32CubeIDE）的工程文件。

安装步骤：从 ST 官网（<https://www.st.com/en/development-tools/stm32cubemx.html>）下载 STM32CubeMX 安装包。解压后运行安装程序，按照向导完成安装。首次启动时，软件会自动检查并下载所需的固件库包。

下图展示了 STM32CubeMX 中芯片引脚配置的工作界面，可以看到芯片的引脚图和配置选项。

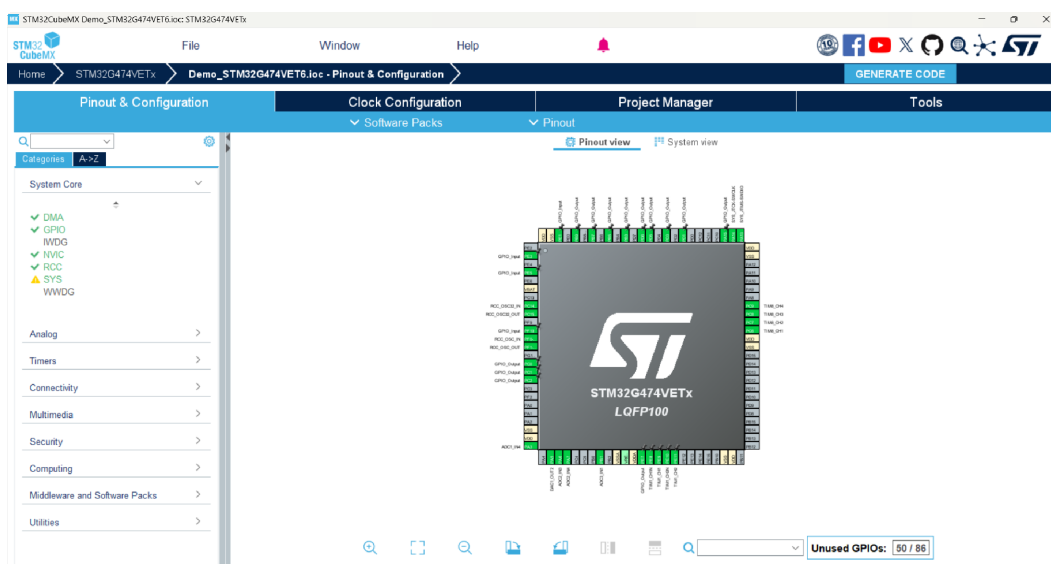


图 3 STM32CubeMX 芯片引脚配置界面

在该界面中，左侧面板可以配置 System Core（系统核心）、Peripherals（外设）、Memory（存储器）、Connectivity（连接）等模块。主区域显示芯片的引脚图，不同颜色的引脚表示不同的功能分配状态。用户可以直观地点击引脚来配置其功能。

3.3.3 MDK-ARM 的安装

MDK-ARM（Microcontroller Development Kit for ARM），又称 Keil MDK，是 ARM 公司官方推荐的嵌入式开发集成环境，主要用于 ARM Cortex-M 系列芯片的裸机程序开发。在 STM32MP157 的双核架构中，MDK-ARM 用于编译 Cortex-M4 核上运行的裸机或 RTOS

程序。

安装步骤：从 Keil 官网 (<https://www.keil.com/>) 下载 MDK-ARM 安装包。安装完成后，需要安装对应芯片系列的器件支持包 (Device Family Pack)。在 Pack Installer 中选择 STM32MP1 系列并安装相应的 DFP 包。

下图展示了 MDK-ARM 中打开项目并进行编译测试的效果。

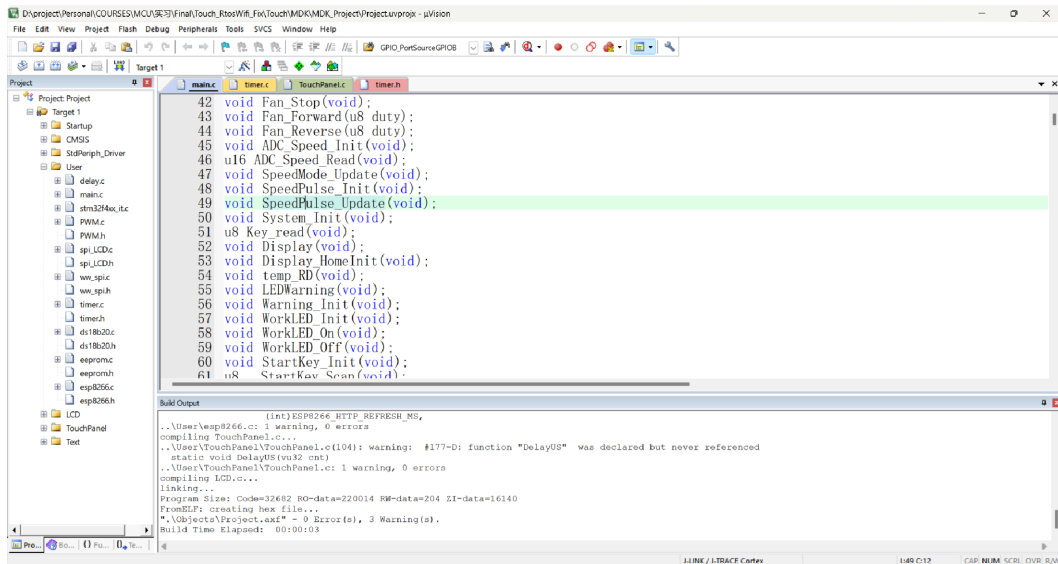


图 4 MDK-ARM 项目编译测试界面

在 MDK-ARM 中，左侧的项目管理器可以浏览项目文件结构，主编辑区支持代码编写和语法高亮。通过 Build 菜单中的"Build Target"或快捷键 F7，可以编译整个项目，底部的 Build Output 窗口显示编译结果和错误信息。

3.3.4 CH340 驱动的安装

CH340 是一款 USB 转串口芯片，广泛应用于开发板的串口通信接口。STM32MP157 开发板通常通过 CH340 芯片将开发板的调试串口 (UART) 转换为 USB 接口，以便通过 USB 线连接到 PC 进行串口调试。

安装步骤：从 CH340 官方下载页面下载 Windows 版本的驱动程序 (CH341SER.EXE)。双击运行安装程序，点击"安装"按钮完成驱动安装。安装完成后，将开发板通过 USB 线连接到 PC，在设备管理器中的"端口 (COM 和 LPT)"下应出现类似"USB-SERIAL CH340 (COMx)"的设备，表示驱动安装成功。记下 COM 端口号，后续使用串口终端工具 (如 PuTTY、XCOM 等) 连接时需要用到。

3.3.5 STM32Programmer 的安装

STM32CubeProgrammer 是 ST 公司推出的 STM32 系列芯片编程工具，用于通过 USB OTG、UART、SPI 等多种接口对 STM32 芯片进行固件烧录和调试。它支持烧录.hex、.bin、.elf 等多种格式的文件，是开发过程中将编译好的程序烧录到开发板的重要工具。

安装步骤：从 ST 官网下载 STM32CubeProgrammer 安装包。解压后运行安装程序，按照向导完成安装。安装过程中会自动安装 USB DFU 驱动，用于通过 USB OTG 接口与开发板通信。

下图展示了 STM32CubeProgrammer 的编程界面。

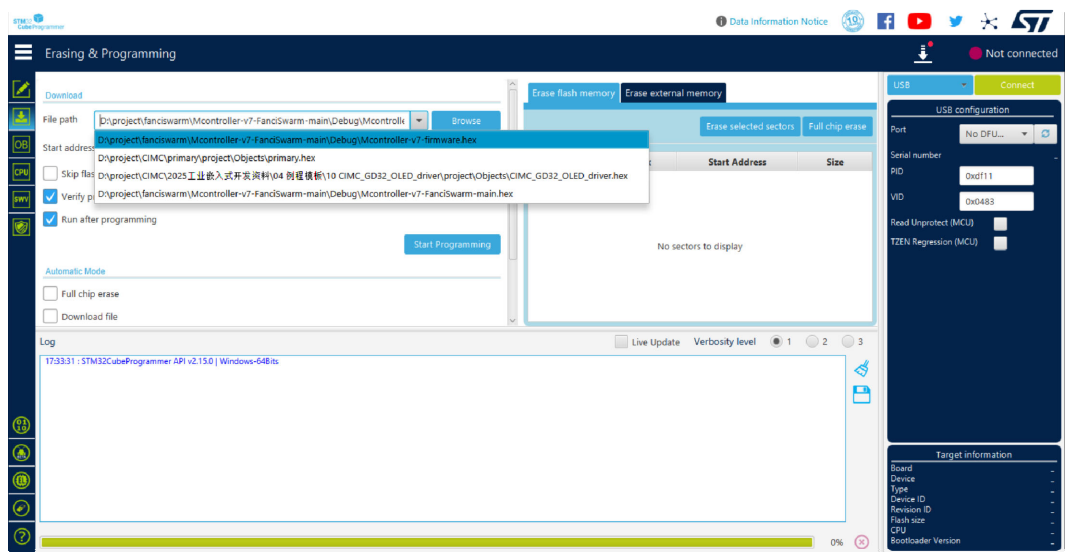


图 5 STM32CubeProgrammer 烧录界面

在该界面中，左侧面板提供了文件路径配置、调试启动、自动复位、全片擦除等功能选项。右侧面板显示目标芯片的信息（芯片型号、Flash 大小、RAM 大小等）。中央区域显示 Flash 存储器的映射信息。当开发板通过 USB 连接到 PC 后，点击"Connect"按钮即可建立连接，然后通过"Download file"按钮将编译好的.hex 或.bin 文件烧录到开发板的 Flash 中。

3.4 安装测试效果总结

完成所有软件工具的安装后，对各工具的可用性进行了逐一验证，测试结果汇总如下：

序号	工具名称	测试结果
1	交叉编译工具链 (arm-none-linux-gnueabi-gcc)	执行 gcc -v 输出版本号 9.2.1，交叉编译 hello.c 生

		成 ARM 可执行文件，测试通过
2	Visual Studio Code	成功启动，C/C++ 扩展和 CMake Tools 扩展安装正常，可正常编辑和浏览项目代码
3	Java 环境 + STM32CubeMX	Java 环境版本正常，STM32CubeMX 成功启动，可配置芯片引脚并生成工程文件
4	MDK-ARM (Keil)	安装完成，项目可正常打开并编译通过，Build Output 无错误
5	CH340 驱动	设备管理器中可见 USB-SERIAL CH340 设备，串口通信正常
6	STM32CubeProgrammer	软件正常启动，可显示目标芯片信息，支持 .hex/.bin 文件烧录

综上所述，所有开发工具均已安装成功并通过功能测试，嵌入式系统开发环境搭建完毕。该环境能够支持后续基于 STM32MP157 开发板的 Linux 驱动开发、U-Boot 移植、内核编译、裸机程序开发等各项实验。

四、实验总结

通过本次课外实验，完成了嵌入式系统开发环境的完整搭建，主要收获如下：

第一，深入理解了交叉编译的概念。交叉编译是在宿主机（X86 PC）上编译生成目标机（ARM 开发板）可执行代码的过程，这是嵌入式开发区别于 PC 软件开发的重要特征。通过安装 `arm-none-linux-gnueabi-gcc` 工具链，掌握了环境变量配置、交叉编译测试等基本技能。同时明确了交叉编译工具链与 `cmake` 的区别：工具链是真正执行编译的编译器，而 `cmake` 是用于管理编译流程的构建系统生成器。

第二，掌握了 Visual Studio Code 在嵌入式开发中的使用方法。VS Code 凭借其丰富的扩展生态（C/C++、CMake Tools、Remote-SSH 等），能够显著提升代码编写和项目管理效率，是嵌入式开发中常用的轻量级 IDE。

第三，完成了开发板配套软件的全面安装。Java 环境是 STM32CubeMX 的运行基础；STM32CubeMX 提供了图形化的芯片配置能力，大大简化了硬件初始化工作；MDK-ARM 用于 Cortex-M4 核的裸机程序开发；CH340 驱动实现了串口调试功能；STM32CubeProgrammer 提供了固件烧录能力。这些工具相互配合，构成了完整的开发工具链。

第四，通过实际测试验证了各工具的功能正确性。交叉编译生成的 ARM 可执行文件、STM32CubeMX 生成的工程文件、MDK-ARM 编译通过的项目等都证明了开发环境搭建成功，为后续的嵌入式 Linux 驱动开发和裸机程序开发奠定了坚实的基础。